

AI Library Platform

NHN 아카데미 KDT 3기



목차

01	프로젝트 소개	_____	3p
02	검색	_____	6p
03	캐싱	_____	9p
04	리뷰	_____	11p
05	텔레그램 & MCP	_____	14p
06	자체 평가 의견	_____	20p

프로젝트 개요

Tech Stack

Backend	Java 21, Spring Boot 3.5
Data Access	Spring Data JPA, QueryDSL
Database	PostgreSQL with pgvector, Redis
AI/LLM	Spring AI, Google Gemini, Ollama, BGE-M3
Batch	Spring Scheduler
Frontend	Thymeleaf

주요 기능

- 키워드 검색
- 벡터 기반 의미 검색
- 하이브리드 검색 (RRF)
- RAG 기반 AI 답변 생성
- 캐싱
- 리뷰 시스템
- Telegram AI Bot & MCP Tool



팀 구성 및 역할

고나영



MCP, Telegram

Telegram Bot
& MCP Tool

김태희



Review

도서 리뷰 관리 서비스
MCP Tool

윤채영



Search

벡터 및 하이브리드 검색
시멘틱 캐싱
MCP Tool

정은선

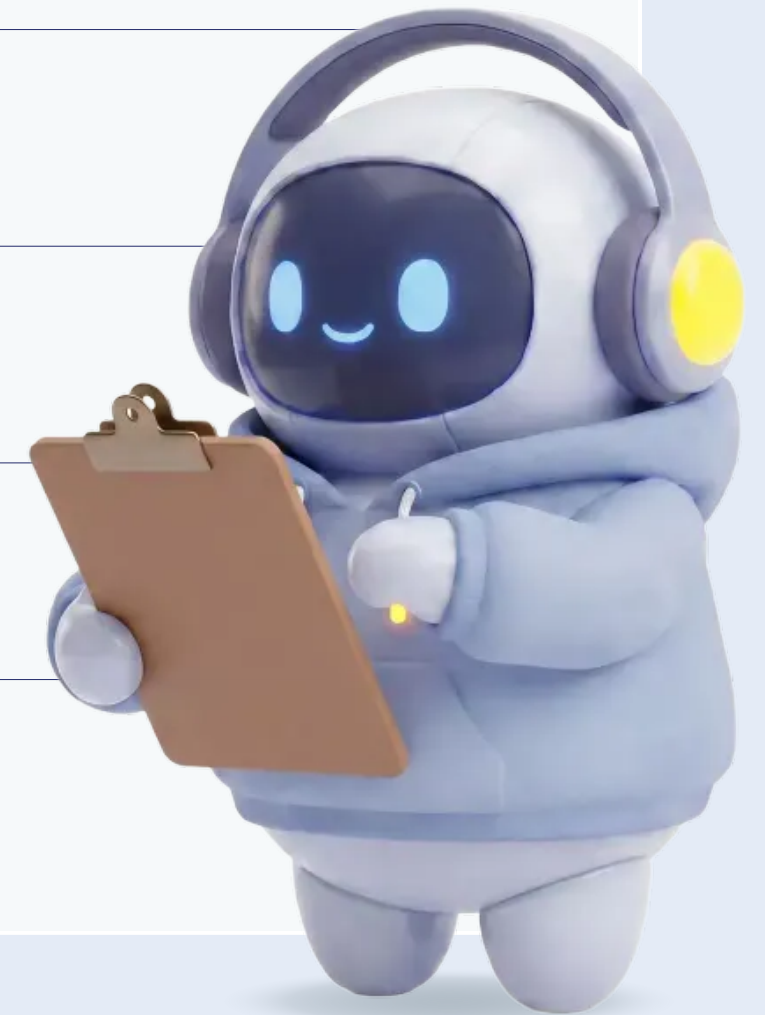


Search

RAG 검색
Redis 캐싱

프로젝트 수행 절차

- ✓ 06.22. → CSV 배치 적재
- ✓ 06.23. ~ 06.24. → 배치 적재 코드 리뷰 및 LIKE/인덱스 기반 검색 구현
- ✓ 06.25. ~ 06.28. → 벡터 검색, RAG 검색, 리뷰, Telegram 1차 구현
- ✓ 06.29. ~ 07.02. → 1차 구현 코드 리뷰 및 2차 구현 (코드 개선 / 성능 향상 / 남은 기능 구현)
- ✓ 07.03. → 발표, 보고서 작성





검색



Keyword

일반적인 키워드 검색과 전문 검색을
결합한 검색 기능



Vector

임베딩한 벡터값을 기반으로
코사인 유사도를 계산해 검색



Hybrid

키워드 검색과 벡터 검색 결과를 조합해
순위를 매기는 하이브리드 검색



RAG

하이브리드 검색의 상위 결과를 기반으로
의미 기반 재정렬 및 답변을 생성하는 검색 증강

검색 유형

Keyword 검색 및 Vector 검색

Keyword

일반적인 키워드 검색과 전문 검색

- DB쿼리의 LIKE 문법을 사용한 일반적인 키워드 검색
- PostgreSQL의 전문검색 함수 (@@)를 이용해 전문 검색기능 추가

Vector

키워드(검색어)의 벡터 및

책 정보 벡터의

코사인 계산을 통한 벡터 검색

- 스케줄러를 통해 5초마다 Book테이블에서 embedding이 NULL인 책을 여러 권 조회 후 일괄 임베딩 후 저장 (배치)
- 저장한 임베딩 벡터값과 키워드의 벡터값을 코사인 유사도 계산을 통해 임계값 필터로 검색

검색 유형

Hybrid, RAG

Hybrid

키워드 검색과 벡터 검색을 조합한 검색

- 키워드 검색과 벡터 검색을 스레드 풀을 이용해 병렬로 실행 (Java CompletableFuture 컨테이너)
- 두 검색이 완료된 후 반환된 결과를 통해 순위를 기준으로 RRF 점수 계산
- RRF 점수 계산
 - $1 / (k + \text{순위} + 1)$ (k는 상수, $k = 60$) => 순위로 인한 값 변동 범위를 줄이기 위해 적당한 크기의 상수 사용

RAG

의미 기반 재정렬 및

답변을 생성하는 검색 증강

- 하이브리드 검색을 통해 RAG용 후보 데이터를 추출
- 선택된 데이터와 리뷰 요약, 평점 정보를 기반으로 컨텍스트 구성
- 구성된 컨텍스트를 활용해 LLM 기반 질의 응답 생성 (검색 증강)



캐싱



키 매칭기반 조회

일반적인 키워드 검색과 전문 검색을
결합한 검색 기능



유사도 기반 조회

임베딩한 벡터값을 기반으로
코사인 유사도를 계산해 검색

캐싱 전략

키 매칭기반 조회

동일 Key 요청 시 Redis 캐시 데이터를 반환하여
조회 성능 향상
@Cacheable을 적용하여 캐시 저장 및 조회를
선언적으로 처리
검색 유형별 Prefix 분리(Keyword, ISBN)로 캐시
Key 충돌 방지 및 관리성 향상

유사도 기반 조회

검색어(KeyWord)를 벡터화하여 검색 결과와 같이 캐
시 저장
검색 시 검색어를 벡터화하여 캐싱되어있는 벡터화 검
색어와 코사인 유사도를 계산해 유사여부 판단
유사시 캐시 데이터 반환, 유사하지 않을 시 새로운 캐
시 저장



리뷰



비동기 메시지 큐

리뷰 등록 시 서버 부하를 방지하기 위해
RabbitMQ 기반의 비동기 파이프라인 구축



비용 최적화

새 리뷰 추가 시 누적 요약하는 방식으로
LLM API 토큰 비용을 절감



동시성 제어

동일 도서에 대한 중복 접근을 막기 위해
Redis 분산 락(Redisson)을 적용하여 정합성을 보장



영속성 확보

PostgreSQL에 요약본을 영구 저장해
데이터 영속성을 확보

도서 리뷰 등록 흐름

Step 01

Step 02

Step 03

Step 04

Step 05

리뷰 등록

사용자가 리뷰를 작성하면
book_review 테이블에
데이터 저장

비동기 작업 큐 적재

내부 이벤트 발행 후
RabbitMQ 큐에
'리뷰 요약 및 통계' 작업
비동기 메시지로 넣음

백그라운드 작업 소비

대기중인 백그라운드
워커 스레드들이
큐에서 작업을 순차적으로
꺼내 처리 시작

동시성 제어

같은 책의 요약 작업
동시 실행 방지를 위해
Redis 분산 락을 시도,
락을 획득한 스레드만
작업 수행

AI 요약

LLM을 호출해 요약을 생성
최종 결과를
book_review_summary
테이블에 저장 후 락 해제

리뷰 화면



리뷰 통계

사용자가 입력한 별점 데이터를 실시간으로 수집하여
평균 별점과 분포도를 보여줌

AI 요약

사용자가 입력한 리뷰를 LLM으로 압축하여
장점, 단점, 총평, 추천 대상을 보여줌



텔레그램 & MCP



Telegram Bot

사용자와 상호작용하는 채팅 인터페이스
Agent로부터 받은 응답 값 format 처리



Agent

자연어를 분석하고 적절한 Tool을 선택
만들어진 응답 값 Telegram에 전달



MCP

외부 기능을 표준 방식으로 연결 및 실행



Open API

공공 도서·도서관 데이터 조회

텔레그램 메시지 흐름

Step 01

Step 02

Step 03

Step 04

Step 05



Telegram

메시지 수신
/agent/chat에 전달

Agent

파라미터 추출
LLM에게 전달
MCP Tool 호출 명령



MCP Server

정보나루 API와 연동
응답 객체 생성

Agent

Tool 호출 결과를
Telegram에 전달



Telegram

Formatter 처리를 통해
요청 유형 별 일관된
응답 메시지 전송

채팅 결과

텔레그램 & MCP 서비스에서 지원하는 기능 및 결과입니다.

Question

- ✓ 대출 가능 도서관 정보 조회 (param : 지역, 세부지역, 도서명)
- ✓ 도서 정보 조회 (param : 도서명, 저자명)
- ✓ 인기 대출 도서 정보 조회 (param : 성별, 연령, 도서명, 지역 등)
- ✓ 이달의 키워드 조회 (param : 날짜)



도서 대출 가능 도서관 조회

광주 남구에서 소년이 온다 대출 가능한 도서관 오전 8:48 ✓

검색 중입니다. 잠시만 기다려주세요. 오전 8:48

대출 가능 도서관 검색 결과
[광주광역시립사직도서관]
주소 : 광주광역시 남구 제중로 56
전화번호 : 062-613-7783
홈페이지 : <https://citylib.gwangju.go.kr/main>
휴관일 : 매월 첫째주, 셋째주 월요일 / 관공서 공휴일, 도서관 사정으로 관장이 지정한 날
운영시간 : 종합자료실 09:00~22:00(주말 17:00), 어린이실 09:00~18:00(주말 17:00), 열람실 07:00~22:00
소장 여부 : 소장 중
대출가능 여부 : 대출 가능

[광주남구청소년도서관]
주소 : 광주광역시 남구 제석로 100
전화번호 : 062-607-2546
홈페이지 : <https://lib.namgu.gwangju.kr/>
휴관일 : 매주 첫째주, 셋째주 월요일 / 법정공휴일
운영시간 : 청소년자료실: 평일 09:00~22:00, 주말 09:00~17:00 / 어린이자료실: 평일 09:00~18:00, 주말 09:00~17:00
소장 여부 : 소장 중
대출가능 여부 : 대출 가능

문제 발생



LLM 파라미터 추출 문제

LLM에게 제공된 DTO에 맞게 파라미터를 추출 / 매핑 하라고 했지만 오류 발생



Tool 선택 오류

비슷한 요구사항을 가진 Tool들로 인해 의도와는 다른 Tool 호출 발생



응답 속도

속도 향상 과정

1. Ollama(qwen2.5:3b-instruct-q4_K_M) > Gemini(2.5-flash-lite)>Gemini(3.1-flash-lite)
2. Prompt 중복된 문장 제거, 핵심 요청 문구 위주로 간소화



응답 값 처리 방식

Telegram과 Agent 서버 분리로 인해 Response 형식에 대해 고민

LLM Prompt & Tool Description

LLM은 사용자의 요청에서 파라미터 추출, Tool 선택, 응답 객체 생성 등을 담당

하지만 원하는 결과값을 내지 못하는 상황 발생

이를 해결하는 과정에서 습득한 Prompt 및 Tool Description 작성 방법

Prompt

- 역할 지정 -> 목표 작성 -> 규칙 -> 출력 형식 -> 예시
- 출력 형식을 구체적으로 지정 (JSON, Markdown 등)
- ✗ 중복되거나 모순되는 지시
- ✗ 추상적인 요청 ("도서관 검색 해줘")

Tool Description

- 역할, 입력 값, 언제 사용하는지, 무엇을 하지 않는지
- 예시를 넣어주는 것도 효과적
- Tool 간의 역할을 명확하게 작성
- ✗ 적절하게, 가능하면, 필요하면 등의 표현

응답 값 처리 방식

01

String

규칙:

1. 무조건 MCP Tool을 사용해서 데이터를 조회하세요.
1. 반드시 등록된 MCP Tool을 사용해서 데이터를 조회하세요.
2. Tool 결과(JSON)를 항목별로 정리해서 자연어로 출력합니다.
3. 결과에는 조회된 정보만 포함합니다.

- Prompt: 응답 형식을 자연어로 지정
- LLM의 응답을 그대로 Telegram에 전달
- 단점 : 응답 형식이 다양함

02

JSON

```
public record ChatResponse( 8개 사용  
    ResultType responseType, 0개  
    JsonNode result 0개의 사용위  
) {
```

- ResultType(LIBRARY, BOOK, LOAN...) 설정
- Tool이 응답한 형식 그대로 JSON으로 반환
- 단점 : Telegram쪽 코드에서 JSON 키 값을 문자열로 지정해줘야 함 (유지보수성)

03

JSON -> DTO

```
private String formatBook(JsonNode result) { 1개 사용 위치 & konayeong  
    List<BookResult> bookResult = mapper.convertValue(result, new TypeReference<List<BookResult>>() {});  
  
    StringBuilder sb = new StringBuilder();  
  
    sb.append("📖 도서 검색 결과");  
  
    for (BookResult book : bookResult) {  
        sb.append("\n[").append(book.bookName()).append("] \n");  
        sb.append("저자 : ").append(book.authors()).append('\n');  
    }  
}
```

- Agent는 그대로 ChatResponse 전달
- Telegram : 받은 JSON을 ResultType별로 정한 DTO로 변환하여 추출

자체 평가 의견

2주간 Ai Library Project를 통해

Telegram & MCP 를 통해 공공 데이터를 활용한 챗봇 기능을 구현할 수 있는 능력을 길렀다.

LLM의 성능은 모델 자체뿐만 아니라 프롬프트와 Tool 설계에 크게 좌우된다는 점을 체감하였다.

직접 개발하지 않았던 검색, 리뷰 또한 코드리뷰 시간을 통해 개념, 흐름을 이해할 수 있었다.

MCP 구현하면서 느낀점

@Tool과 @McpTool의 역할과 차이를 이해하여 적용할 수 있었고, Tool이 점차 추가되면서 발생한 LLM의 Tool 선택 및 호출 문제를 분석하고 개선하는 과정에서 Tool Description이 LLM의 의사결정에 미치는 영향을 직접 경험하며 효과적인 Description 작성 방법을 학습하였다

또한 정보나루 Open API를 활용해 Tool을 구현하면서, 단순히 LLM에 모든 처리를 맡기는 것이 아니라 LLM과 애플리케이션 로직의 역할을 적절히 분리하고 조합하는 설계 방식을 익힐 수 있었다.



Agent 구현하면서 느낀점

Tool 선택을 담당하는 LLM의 프롬프트를 설계할 때 사용자의 의도를 정확히 이해하여 적절한 Tool을 호출하도록 만드는 것이 예상보다 어려웠다. 특히 Tool의 역할이 유사하거나 개수가 증가할수록 의도와 다른 Tool이 선택되는 문제가 발생했으며, 이를 해결하기 위해 Tool Description과 프롬프트를 반복적으로 개선하면서 LLM이 판단에 활용하는 정보를 어떻게 설계해야 하는지 학습할 수 있었다.

Thank You

AI를 이해하는 것을 넘어, 실제 서비스로 구현하는 경험을 담았습니다.

